Étude de cas 5 : Maintenance prédictive

Jalon 4 : L'histoire de la pompe

chapitre 4 - RNN, LSTM et GRU

M2 SDIA - Advanced Artificial Intelligence

Année universitaire 2026–2027

Objectif du jalon

Prédire la RUL à partir d'une fenêtre de cycles avec des modèles récurrents, et traiter les historiques trop courts.

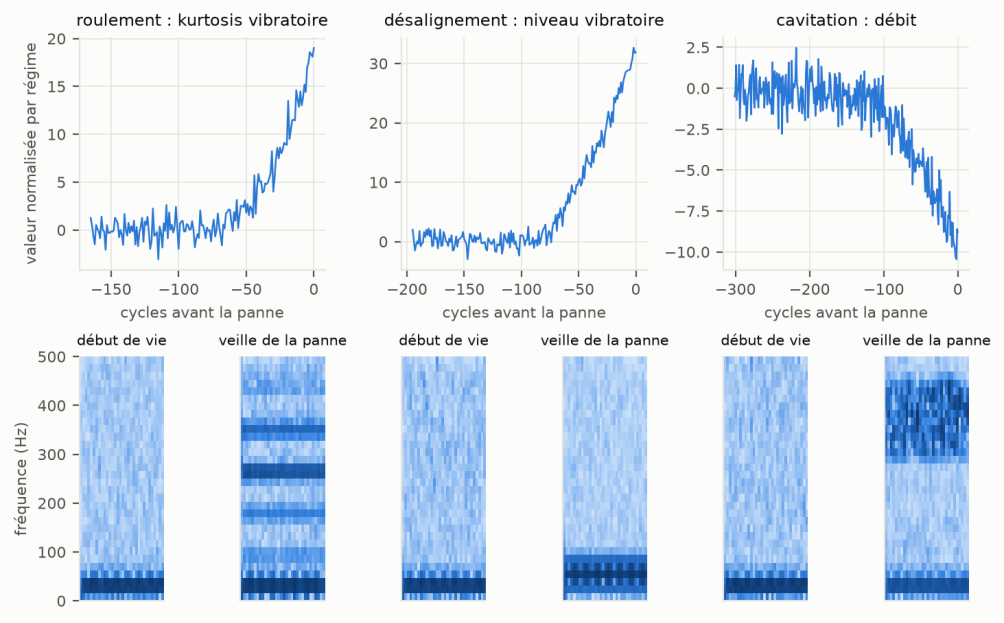
Jalon autonome : ce jalon ne suppose pas d'avoir terminé les jalons précédents. Il s'appuie uniquement sur le notebook de démarrage `uc5_demarrage.ipynb`, le dossier `data/` et le code de démarrage de la section 3. Vous pouvez repartir de votre propre travail antérieur, mais rien ne l'exige.
Environnement : Python 3, TensorFlow/Keras, scikit-learn ; Google Colab avec GPU recommandé.

1 Contexte

Un site minier fictif exploite une flotte de pompes de lavage. Une panne imprévue arrête la ligne de production ; un remplacement trop précoce gaspille une pièce encore en état. Chaque pompe transmet une fois par jour (un *cycle*) huit mesures agrégées (températures, courant, pression, débit, vitesse, niveau et impulsivité des vibrations) ainsi qu'un relevé vibratoire d'une seconde, transformé en spectrogramme. Trois modes de défaillance existent : usure de roulement, désalignement et cavitation. Les capteurs dépendent aussi du **régime de charge** (60, 80 ou 100 %), qui change d'un jour à l'autre.

Un nouveau modèle de pompe (type B, deux fois plus rapide et plus puissant) vient d'être installé ; seules dix pompes de ce type ont déjà été suivies jusqu'à la panne.

L'étude de cas en cinq jalons : un jalon par chapitre du cours, rendu après le chapitre correspondant, puis une synthèse défendue à l'oral en fin de module (note de décision, prédictions sur le test caché, soutenance).



2 Données et cibles de ce jalon

Fichier	Contenu
`uc5_flotteA.npz`	type A : 70 pompes suivies jusqu'à la panne (entraînement), 15 (validation), 30 pompes dont l'historique est coupé avant la panne (test)

Pour chaque cycle : numéro de pompe et de cycle, régime de charge, huit capteurs, spectrogramme 32×32 (temps sur une seconde en abscisse, fréquences de 0 à 500 Hz en ordonnée) ; pour les historiques complets, la RUL et le mode de défaillance. **Découpage imposé par pompe** : les cycles d'une même pompe ne sont jamais répartis entre deux ensembles ; faites de même pour vos plis de validation croisée. Les données sont simulées.

Cibles du cahier des charges concernées

Exigence	Mesure (ensemble de test)	Cible
Durée de vie restante, type A	RMSE sur la RUL plafonnée à 125 cycles	≤ 23 cycles
Coût des erreurs, type A	score asymétrique	$\leq 1\,200$
Anticipation des pannes	part des pompes à 30 cycles ou moins de la panne qui sont signalées	$\geq 0{,}90$
Fausses alertes	part des pompes à plus de 60 cycles de la panne qui sont signalées	$\leq 0{,}10$
Déploiement	taille du modèle	≤ 2 Mo

La **RUL** est le nombre de cycles avant la panne ; elle est plafonnée à 125 cycles. Une pompe est **signalée** lorsque sa RUL prédite est inférieure à 30 cycles. Le **score asymétrique** (challenge NASA C-MAPSS) vaut, avec $d_i = \hat{y}_i - y_i$: $s = \sum_i (e^{-d_i/13} - 1)$ si $d_i < 0$ et $s = \sum_i (e^{d_i/10} - 1)$ sinon (fonction `score_nasa` du notebook). Ces cibles décrivent le

système *final* : elles ne sont pas exigées à la fin de ce jalon. Mesurez-vous par rapport à elles et expliquez l'écart ; un écart argumenté vaut mieux qu'une cible atteinte sans explication.

3 Point de départ

Le notebook de démarrage fournit les données normalisées par régime, les métriques et le **MLP de référence sur un seul cycle** (section 3), comparateur de ce jalon.

Le code ci-dessous construit les fenêtres de W cycles (une par cycle observé, avec deux modes de remplissage) et une fonction qui mesure la norme du gradient de chaque variable.

Code de démarrage (fichier `uc5_jalon4_depart.py`)

```
# Étude de cas 5 - Jalon 4 : code de démarrage
# À exécuter à la suite du notebook de démarrage, exécuté en entier.
import tensorflow as tf
from numpy.lib.stride_tricks import sliding_window_view

def sequences(df, W, remplissage="repetition"):
    # une fenêtre (W cycles x 9 variables) par cycle observé ; au début de
    # l'historique : répétition du 1er cycle, ou zéros (layers.Masking)
    X, y, dernier = [], [], []
    for _, g in df.sort_values(["pompe", "cycle"]).groupby("pompe"):
        v = g[CAPTEURS + ["charge"]].values.astype("float32")
        if remplissage == "repetition":
            debut = np.repeat(v[:1], W - 1, axis=0)
        else:
            debut = np.zeros((W - 1, v.shape[1]), "float32")
        v = np.vstack([debut, v])
        X.append(sliding_window_view(v, W, axis=0).transpose(0, 2, 1))
        dernier.append(np.arange(len(g)) == len(g) - 1)
        if "rul" in g:
            y.append(np.minimum(g.rul.values, RUL_MAX) / RUL_MAX)
    y = np.concatenate(y).astype("float32") if y else None
    return np.concatenate(X), y, np.concatenate(dernier)

W = 30
X_seq, y_seq, _ = sequences(NA["train"], W)
Xv_seq, yv_seq, _ = sequences(NA["val"], W)
Xt_seq, _, fin_t = sequences(NA["test"], W)    # prédire sur Xt_seq[fin_t]
print(X_seq.shape, Xt_seq[fin_t].shape)

def normes_gradient(model, X, y):
    # norme du gradient de chaque variable, sur un lot
    with tf.GradientTape() as tape:
        perte = keras.ops.mean((model(X, training=True)[:, 0] - y) ** 2)
    grads = tape.gradient(perte, model.trainable_variables)
    return {v.path: float(tf.norm(g))
            for v, g in zip(model.trainable_variables, grads)}
```

4 Travail demandé

Jalon 4 : L'histoire de la pompe

Notions mobilisées : réseaux récurrents, many-to-one, rétropropagation dans le temps, disparition du gradient, écrêtage du gradient, LSTM, GRU, remplissage et masquage.

1. Prédire la RUL à partir d'une fenêtre de W cycles ($W \in \{10, 30, 60\}$) avec un LSTM puis un GRU ; comparer au MLP de référence.
2. Historiques plus courts que W : comparer le remplissage par répétition et le remplissage par zéros avec masquage (`layers.Masking`) sur les pompes récentes.
3. Tracer la norme des gradients pendant l'entraînement d'un RNN simple et d'un LSTM sur des fenêtres longues ; tester l'écrêtage du gradient.
4. Facultatif : un CNN appliqué à chaque spectrogramme, suivi d'un LSTM sur la suite des cycles.

Vérification théorique (à la main, puis vérifiée par le code)

1. Nombre de paramètres d'un LSTM à 32 unités recevant 9 variables par cycle, puis d'un GRU de même taille (formulation de Keras).
2. Combien de fenêtres complètes (pas de 1) une pompe de 275 cycles fournit-elle avec $W = 30$? Et avec le remplissage du code de démarrage, qui produit une fenêtre par cycle observé ?

À justifier dans votre page de décisions

La longueur de fenêtre retenue ; le traitement des historiques courts.

5 À rendre à la fin du chapitre

Livrables du jalon 4

- le notebook du jalon, exécuté et reproductible (graine fixée, versions indiquées) ;
- le journal d'expériences à jour (fonction `journaliser` du notebook de démarrage, avec `jalon=4`), y compris les essais infructueux ;
- une page de décisions : tableau des résultats, choix retenus et justifications, réponses aux vérifications théoriques.

Protocole : tous les réglages se font sur la validation ; l'ensemble de test ne sert qu'une fois, à la fin du jalon, pour le modèle retenu. Ce jalon alimente la note de décision finale et la soutenance.